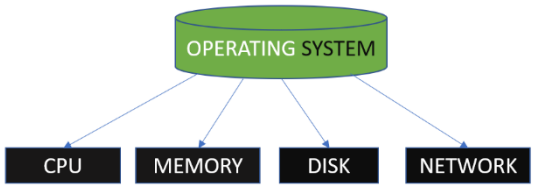
Main Resources of Operating System

CPU (Central Processing Unit)

- Executes all instructions
- OS schedules processes for CPU usage
- Allocates CPU time to multiple programs

Memory (RAM)

- Stores data and programs currently in use
- Provides fast access to running processes
- OS allocates and deallocates memory



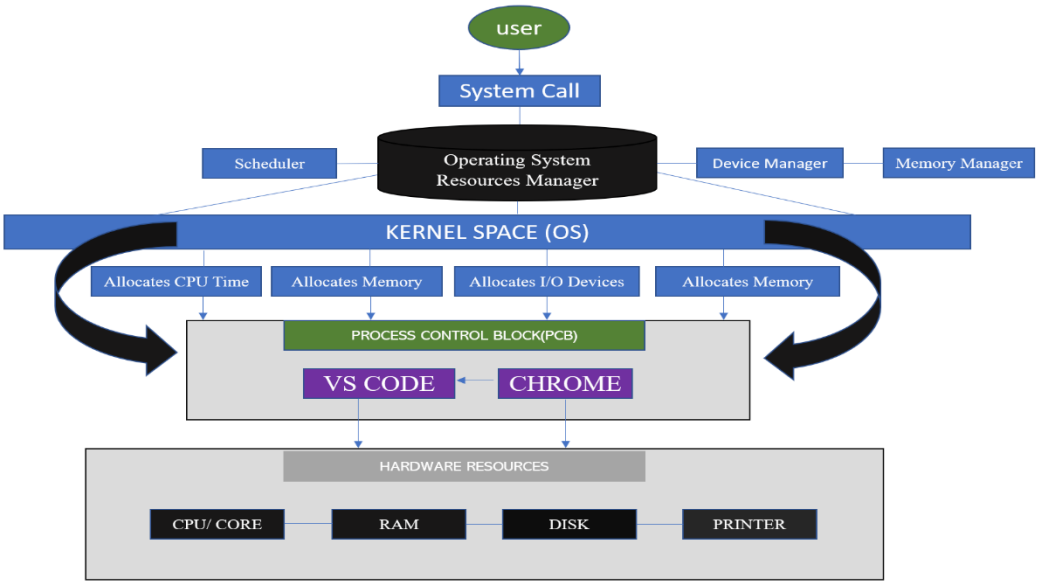
Disk (Storage)

- Stores data permanently (HDD/SSD)
- Keeps files, applications, and OS
- OS manages file system
- Handles read and write operations

Network

- Connects computer to other systems
- OS manages data transmission

Diagram and Explanation:



Step 1: Process Makes a Request

A process cannot directly use hardware.
So, it **requests resources from the Operating System (OS)** using a **system call**.

Example:
VS Code requests:

- CPU time to run
- Memory to store data
- Disk access to open/save files

Step 2: Request Goes to the OS (Kernel)

The request enters the **kernel (core part of the OS)**.
The OS acts as a **resource manager**.

It decides:

- Which process should get resources
- How much resource should be given

Step 3: OS Keeps Track of Processes

The OS stores all process information in a **Process Table (PCB)**.

This includes:

- Process state (running, waiting, etc.)
- Memory details
- CPU usage

Step 4: CPU Allocation (Scheduling)

The **scheduler** decides:

- Which process gets the CPU
- For how long (time slice)

This is called **CPU scheduling**.
Multiple processes share the CPU using **time-sharing**.

Step 5: Memory Allocation

The **memory manager**:

- Allocates a portion of RAM to the process
- Ensures processes do not interfere with each other

Step 6: I/O Allocation

The **device manager**:

- Provides access to devices (disk, keyboard, printer)
- Controls input/output operations

Step 7: Process Execution

After getting all resources:

- The process starts executing
- It uses CPU, memory, and I/O devices

Step 8: Resource Release

When the process finishes:

- It releases all resources
- The OS makes them available for other processes

Main Properties of Concurrency

Atomic Operation

- A sequence of operations that is **indivisible**
- Cannot be interrupted by other processes
- No intermediate state is visible

Critical Section

- Part of code where **shared resources are accessed**
- Only **one process** can execute at a time
- Prevents data inconsistency

Deadlock

- Two or more processes **wait for each other forever**
- None can proceed
- System gets stuck

Livelock

- Processes keep **changing states continuously**
- But **no actual work is done**
- Similar to deadlock but still active

Mutual Exclusion

- Only **one process at a time** can enter critical section
- Prevents conflicts in shared resources

Race Condition

- Multiple processes access shared data **simultaneously**
- Final result depends on **execution timing**
- Can cause incorrect output

Starvation

- A process waits **indefinitely**
- Scheduler never gives it CPU
- Even though it is ready to run

Ref: Main book , class ppt slides and Ai